Product Requirement Document (PRD) & Blueprint: On-Page SEO Competitor Analyzer

---

This document details the architecture, feature specs, tech stack, data scraping pipeline, and execution roadmap for building a non-AI, high-performance SEO Competitor Analysis Web Application.

1. Executive Summary & Core Value Proposition

The goal of this project is to build a fast, browser-accessible, non-AI SEO tool that empowers content creators, bloggers, and SEO professionals to conduct instant competitor on-page audits. By inputting single or multiple URLs, the application extracts critical structural metadata, heading hierarchy, image tags, word counts, and keyword density metrics without relying on expensive LLM APIs.

Core Value Propositions

- **Zero AI Overhead:** Operates purely on high-speed HTML parsing algorithms, eliminating latency and token costs.
- **Side-by-Side Comparison:** Allows multi-URL audits in a single unified dashboard, solving the single-URL limitation found in most browser extensions.
- **Actionable Content Briefs:** Automatically generates exportable outline templates based on aggregated competitor heading structures.
- **Privacy-First & Free Access:** Provides friction-free access without mandatory registration for basic tier searches.

2. Core Functional Features Specification

Feature Name	Technical Functionality	Output Delivered to User
Single & Multi-URL Scraper	Fetches raw HTML DOM via asynchronous HTTP requests with user-agent	Raw data extraction within < 3 seconds per batch of 5 URLs.

Feature Name	Technical Functionality	Output Delivered to User
	rotation and proxy handling.	
Heading Tree Extractor	Parses <h1> through <h6> tags using DOM selection logic (e.g., Cheerio / BeautifulSoup).	Collapsible tree structure showing exact heading hierarchy and nesting order.
Word Count & Readability Engine	Strips HTML tags, scripts, and navigation/footer blocks, then tokenizes body content.	Clean body word count, reading time estimate (200 words/min), and character count.
Image & Media Auditor	Scans <img> nodes to measure total count, missing alt attributes, and WebP format usage.	Total images count, list of images lacking Alt text, and asset optimization tips.
Keyword Density Calculator	Tokenizes cleaned text into 1-gram, 2-gram, and 3-gram frequencies while removing common stop-words.	Interactive table sorting terms by frequency and percentage density.
Side-by-Side Comparison Matrix	Aligns data from up to 5 URLs in a unified comparison grid.	Comparative table of Word Count, Heading Count, Image Count, Title Length, and Meta Description.
Exportable Content Outline Generator	Compiles missing H2/H3 headings across top competitors into a consolidated text block.	One-click downloadable Markdown / Plain Text content brief for writers.

3. Technical Architecture & Tech Stack

The system is designed as a decoupled modern web application consisting of a lightweight frontend, a high-throughput backend scraper service, and an in-memory caching layer.

Recommended Tech Stack Options

Layer	Option A (Node.js Ecosystem)	Option B (Python Ecosystem)
Frontend Framework	React.js / Next.js (Tailwind CSS)	Vue.js / Nuxt.js (Tailwind CSS)
Backend API	Node.js (Express.js or Fastify)	Python (FastAPI or Flask)
Parsing Library	Cheerio / Puppeteer	BeautifulSoup4 / lxml / Playwright
Caching & Rate Limiting	Redis + BullMQ	Redis + Celery
Hosting & Serverless	Vercel (Frontend) + Render/Railway (Backend)	Vercel (Frontend) + DigitalOcean / AWS EC2 (Backend)

System Processing Workflow

1. **Request Trigger:** User inputs target URLs on the frontend dashboard.
2. **Cache Check:** Backend queries Redis cache to see if the URL was parsed within the last 24 hours.
3. **HTML Fetching:** If uncached, backend dispatches an HTTP request with custom User-Agent headers to fetch the target HTML.
4. **DOM Sanitization & Parsing:** Removing boilerplate elements (<nav>, <footer>, <script>, <style>) to isolate primary article content.
5. **Data Extraction:** Extracting Metadata, Headings, Word Frequency, and Image attributes.
6. **Response Delivery:** Transmitting JSON payload back to the frontend for UI rendering and

storing results in Redis.

4. Mathematical Formulas & Logic Implementation

All core analytics rely on deterministic mathematical logic rather than statistical machine learning models.

4.1 Keyword Density Formula

Keyword density is calculated by dividing the frequency of a specific term by the total clean body word count of the document:

```
Keyword Density (%) = ( Keyword Frequency / Total Body Word Count ) * 100
```

4.2 Stop-Word Filtering Logic

To avoid cluttering density metrics with non-essential terms (e.g., "the", "and", "is", "in"), the tokenizer filters words against a standardized stop-word list before computing 1-gram, 2-gram, and 3-gram frequencies.

4.3 Main Content Extraction Logic (Pseudocode)

```
function extractMainContent(htmlString):
    DOM = parseHTML(htmlString)

    // Remove non-content elements
    DOM.remove('script', 'style', 'nav', 'footer', 'header', 'iframe', 'aside')

    // Identify primary content container (article, main, or highest paragraph
density container)
    mainContainer = DOM.find('article') OR DOM.find('main') OR DOM.find('body')

    cleanText = mainContainer.text().replace(/\s+/g, ' ').trim()
    wordsArray = cleanText.split(' ')

    return {
        cleanText: cleanText,
        wordCount: wordsArray.length
    }
```

5. Monetization Strategy & Business Model

To ensure sustainable revenue without discouraging new users, a multi-tiered monetization

strategy will be implemented:

1. **Freemium Subscription Model (SaaS):**
    - **Free Tier:** Up to 5 single-URL searches per day, basic Heading Tree view, web display only.
    - **Pro Tier ($9/month):** Unlimited searches, batch multi-URL processing (up to 10 URLs at once), PDF/CSV export, and outline generator.
2. **Display Monetization (Google AdSense / Ezoic):**
    - Strategic banner placements on the free results page. Clean layout ensured to prevent UI degradation.
3. **Contextual Affiliate Marketing:**
    - Automated affiliate recommendations based on audit outputs (e.g., recommending hosting partners if page load times are high, or copywriting software links).

6. Step-by-Step Execution Roadmap

Phase	Milestone Focus	Key Deliverables	Estimated Timeline
Phase 1	Core Scraper & Logic	Build Node.js/Python scraper script for Title, Meta, Headings, Word Count, and Keyword Density calculation.	Week 1 - Week 2
Phase 2	API & Caching Layer	Set up REST/GraphQL endpoints, integrate Redis caching to minimize server load and block risk.	Week 3
Phase 3	Frontend Development	Develop responsive UI/UX using React/Tailwind CSS,	Week 4 - Week 5

Phase	Milestone Focus	Key Deliverables	Estimated Timeline
		including comparison tables and charts.	
Phase 4	Features & Exports	Implement PDF/CSV export functions, Outline Generator, and Stop-Word filter customizer.	Week 6
Phase 5	Testing & Deployment	Conduct load testing, proxy integration for anti-bot bypass, deploy on Vercel + Railway.	Week 7
Phase 6	Monetization & Marketing	Integrate Stripe payment system, add AdSense code, and publish introductory SEO blog posts.	Week 8 Onward